

CL-TAV algorithm specification package

CL-TAV specification (CR-2026-014 / DD-036)

This package typesets the CL-TAV algorithm specification. The main text gives the top-level session and the three core mechanisms (test selection, timed observation interpretation, history-compatible update). The appendix gives the S2/S6/S7–S8 coordination wrappers, the typed records, the responsibility map and the unresolved-kernel register. It is not a second program and selects no solver technology. Step labels S0–S10 are the stable bilingual mapping; typeset line numbers are not cited.

Algorithm 1: CL-TAV top-level session (ALG-CLTAV-01)

```
Require:  $H_0 = \{h_{\text{normal}}\} \cup H_{\text{single}}$ ; SessionContext  $\Gamma$  (unique state); HistoryHandle  $\eta$ ; exactly one resource mode  
BUDGET( $c_{\min}, B$ ) xor ROUNDS( $K_{\max}$ ); action library  $L$ ; uncertainty reference  $U$   
Ensure: stop class, final  $H$ ,  $\eta$ , session trace; singleton  $h_{\text{normal}}$  is not protocol PASS  
// S0 initialize state, candidates, history, resource, retry and the uncertainty reference  
1  $(\Gamma, \eta, H) \leftarrow \text{Initialize}(H_0, L, U, \text{resourcePolicy})$ ;  
2 while true do  
    // S1 ENTRYSTOP: maps to the existing A2--A5 / P1--P5 decision table (Appendix C)  
3  $\text{stop} \leftarrow \text{IF-RESOURCE-STOP}(\Gamma, H, \eta)$  with IF-EQUIV( $\Gamma, H, \eta$ );  
4 if  $\text{stop} \neq \text{undecided}$  then return Finish( $\text{stop}, \Gamma, \eta, H$ );  
    // S2 typed prediction; the gap tag is handled before any projection (ALG-CLTAV-02)  
5  $\text{pred} \leftarrow \text{PredictCurrent}(\Gamma, \eta, H, U, L)$ ;  
6 if  $\text{pred.status} = \text{GAP}$  and not (RecoverEligible( $\Gamma, L$ ) or PrepEligible( $\Gamma, L$ )) then return Finish( $\text{pred.reason}, \Gamma, \eta, H$ );  
    // S3 selection and admission; a tagged exit or spec error returns with its reason  
7  $\text{dec} \leftarrow \text{SelectAndAdmit}(\Gamma, \eta, H, L, \text{pred})$ ;  
8 if  $\text{dec.kind} \in \{\text{EXIT}, \text{SPEC-ERROR}\}$  then return Finish( $\text{dec.reason}, \Gamma, \eta, H$ );  
    // freeze the immutable select snapshot  $\xi$  (actionId, kind, qUsedAtSelect, historyVersion,  
    classesUsedAtSelect, uncertaintyRef) after the final action choice (ALG-CLTAV-05)  
9  $\xi \leftarrow \text{SelectSnapshot}(\text{dec.actionId}, \text{dec.kind}, \Gamma.\text{currentSummary}, \eta.\text{version}, \text{dec.classesUsed}, U)$ ;  
    // S4 charge once; retry cap  $\neq$  budget exhaustion  
10  $\Gamma \leftarrow \text{ChargeOnce}(\Gamma, \xi)$ ;  
    // S5 EXECUTEANDRECORD: execute the snapshot action exactly once  
11  $\text{er} \leftarrow \text{IF-EXECUTE-RECORD}(\Gamma, \xi)$ ;  
    // S6 INTERPRETOUTCOME: ALG-CLTAV-03 coordinates; ALG-CLTAV-06 interprets timing  
12  $z \leftarrow \text{InterpretOutcome}(\text{er}, \xi, \Gamma, \eta, U)$ ;  
    // S7--S8 RESOLVEOUTCOME: only COMMIT reaches S9  
13  $(\Gamma, \eta, \delta) \leftarrow \text{ResolveOutcome}(z, \xi, \Gamma, \eta)$ ;  
14 if  $\delta.\text{control} = \text{STOP}$  then return Finish( $\delta.\text{reason}, \Gamma, \eta, H$ );  
15 if  $\delta.\text{control} = \text{RETRY}$  then continue;  
    // S9 COMMITCOMPATIBLEUPDATE: guarded history-compatible update and successor-summary commit  
16  $(\eta, H, \Gamma) \leftarrow \text{CommitCompatibleUpdate}(\xi, \delta.\text{outcome}, \Gamma, \eta, H)$ ;  
    // S10 POSTUPDATESTOP: existing post-update stop priority; otherwise the next iteration is S1  
17  $\text{stop} \leftarrow \text{IF-RESOURCE-STOP}(\Gamma, H, \eta)$  with IF-EQUIV( $\Gamma, H, \eta$ );  
18 if  $\text{stop} \neq \text{undecided}$  then return Finish( $\text{stop}, \Gamma, \eta, H$ );
```

Algorithm 2: SelectAndAdmit: test selection and resource admission (ALG-CLTAV-05, S3)

Require: Γ ; η ; current H ; action library L ; typed $pred$ (PredictionResult)
Ensure: a Decision record: kind ACTION / EXIT / SPEC-ERROR with actionId, actionKind, classesUsed and reason; no fabricated action on an exit

- 1 $A \leftarrow \{t \in L : \text{Enabled}(t, \Gamma)\} \cap \{t : \text{Affordable}(t, \Gamma)\}$;
- 2 $C_t \leftarrow \{H \cap C : C \in pred.\text{classesByTest}[t], H \cap C \neq \emptyset\}$ for each TEST t ;
// a backend gap is a tagged error, not an empty class collection and not a score of zero
- 3 if $pred.\text{status} = \text{GAP}$ for an enabled TEST t and $C_t = \emptyset$ then return SPEC-ERROR with $pred.\text{reason}$;
- 4 ;
- 5 $(k, S, admit) \leftarrow \text{IF-SELECT-ADMIT}(A, \Gamma.\text{currentSummary}, \{C_t\}, \Gamma, \eta, H)$;
// the IF-SELECT-ADMIT contract returns the exclusive kind TEST / PREP / RECOVER / ADMIT; the rules below make the final action deterministic
- 6 if $k = \text{ADMIT}$ then
- 7 | return EXIT with the existing Admit A2–A5 reason;
- 8 else if $k = \text{RECOVER}$ then
- 9 | return ACTION(eligible Recover, RECOVER, $\cdot, \cdot$) granting the session’s UNKNOWN-recovery eligibility;
- 10 else if $k = \text{PREP}$ then
- 11 | $p^* \leftarrow \arg \min_{p \in S} (\text{cost}(p), \text{id}(p))$; return ACTION(p^* , PREP, $S, \cdot$);
// TEST branch: distinguishing value, then one-step minimax, then the existing cost/id tie-break
- 12 distinguishing $\leftarrow \{t \in S : \exists C \in C_t, 0 < |C| < |H|\}$;
- 13 if distinguishing = $\emptyset$ then return EXIT with the admission reason; never $\arg \min \emptyset$;
- 14 ;
- 15 $s(t) \leftarrow \max_{C \in C_t} |C|$ for each $t \in \text{distinguishing}$;
- 16 $t^* \leftarrow \arg \min_t (s(t), \text{cost}(t), \text{id}(t))$ over distinguishing;
// overlapping classes remain legal; worst value = $|H|$ is admissible but not distinguishing value
- 17 return ACTION(t^* , TEST, $C_{t^*}, \cdot$);

Algorithm 3: InterpretTimedObservation: timed observation interpretation (ALG-CLTAV-06, S6 kernel)

Require: ExecutionResult er ; SelectSnapshot ξ ; Γ ; η ; declared match policy Match_r ; uncertainty contract U
Ensure: a TimingResult: verdict in {PASS, FAIL, INCONCLUSIVE, ERROR}; matching evidence; validity/error; the data needed for a conservative compatible interpretation

// event validity and one instance-ownership result; T2 and the no-response rule consume this same result, not two independent matches

- 1 owns $\leftarrow \text{IF-OBS-INTERPRET}(\Gamma, er.\text{record})$;
- 2 if owns is not valid then
- 3 | return TimingResult(verdict = ERROR); the candidate set is unchanged;
// match by Match_r type, correlation key, pairing policy and order; a cancellation or superseded instance does not claim a later response
- 4 pair $\leftarrow \text{Match}_r(\text{owns}, \xi)$;
- 5 if pair is ambiguous then return TimingResult(verdict = ERROR); no silent pick;
- 6 ;
- 7 if pair is a legitimate trigger/response pair then
- 8 | $M \leftarrow [\hat{\Delta}t - \varepsilon, \hat{\Delta}t + \varepsilon] \cap D_r$; // measurement interval M , distinct from the verdict and from the candidate set
- 9 | if $M \subseteq N_r$ then
- 10 | | return TimingResult(verdict = PASS, evidence = pair, M);
- 11 | | else if $M \cap N_r = \emptyset$ then
- 12 | | return TimingResult(verdict = FAIL, evidence = pair, M);
- 13 | return TimingResult(verdict = INCONCLUSIVE, evidence = pair, M); open/closed endpoints preserved;
// a missing response forms a no-response only after a still-active instance reaches its deadline; this round not seeing it is not a timeout
- 14 if the obligation is still active and expired then
- 15 | let M be the earliest admissible horizon; return the FAIL/INCONCLUSIVE/ERROR verdict by the same M vs N_r rule;
- 16 return TimingResult(verdict = ERROR); a missing pair is not a point observation;

Algorithm 4: CommitCompatibleUpdate: history-compatible update and summary commit (ALG-CLTAV-07, S9)

Require: SelectSnapshot ξ ; normalized Outcome z from the resolving module; Γ ; η ; H
Ensure: successor η' , H' and, under the guard, the committed Γ' .currentSummary; or a named spec error
 // only reached under the existing S9 guard (a valid observation class or a confirmed successor summary); ERROR is not fed here as an empty observation

```

1 if  $z.I_z$  is a valid observation class then
2    $H' \leftarrow H \cap z.I_z$ ; // intersection, never  $H \leftarrow z.I_z$ ; no resurrection of a removed hypothesis
3 else
4    $H' \leftarrow H$ ; // a confirmed Prep with no valid  $I_z$  advances history but preserves  $H$ 
5  $(\eta', H') \leftarrow \text{IF-HIST-UPDATE}(\eta, H, \xi.\text{actionId}, \xi.\text{qUsedAtSelect}, \xi.\text{classesUsedAtSelect}, z.I_z, z.\text{postSummary})$ ;
  // IF-HIST-UPDATE reads the select-time snapshot, not the post-effect summary; whole-history compatibility, not
  // per-round feasibility
6 if  $\Gamma.\text{qStatus}$  is KNOWN and  $z.\text{summaryConfirmed}$  is true and  $z.\text{postSummary}$  is confirmed then
7    $\Gamma'.\text{currentSummary} \leftarrow z.\text{postSummary}$ ; // this is the sole commit of the confirmed successor summary
8 return  $\eta', H', \Gamma'$ ;
```

Algorithm 5: PredictCurrent: current-observation prediction wrapper (ALG-CLTAV-02, S2)

Require: SessionContext Γ ; HistoryHandle η ; current H ; uncertainty reference U ; action library L
Ensure: PredictionResult: status OK with classesByTest and metadata, or status GAP with a reason; the session's own η is read, not rebuilt

```

1  $raw \leftarrow \text{IF-PRED-OBS}(\Gamma, \eta, H, U, L)$ ;
  // branch on the backend result tag BEFORE projection; an error sentinel is not a set of observation classes
2 if  $raw.\text{tag} = \text{GAP}$  then
3   return PredictionResult(status = GAP, reason =  $raw.\text{reason}$ ); // the gap stays TEST-scoped and does not forbid
  // eligible Prep/Recover
4  $classes \leftarrow \text{ProjectOntoCurrentH}(raw.\text{classes}, H)$ ; // an empty projection from an excluded hypothesis is not a
  // surviving score = 0 class
5 return PredictionResult(status = OK, classesByTest =  $classes$ , historyVersionUsed =  $\eta.\text{version}$ , uncertaintyRef =  $U$ );
```

Algorithm 6: InterpretOutcome: execution and observation coordination (ALG-CLTAV-03, S6)

Require: ExecutionResult er ; SelectSnapshot ξ ; Γ ; η ; uncertainty reference U
Ensure: a total Outcome z : every field initialized; skipped confirmation is NOT-EVALUATED, not an evaluated failure
 // total record constructor (fields in Appendix A); nothing is left uninitialized

```

1  $z \leftarrow \text{NewOutcome}()$ ; // all Outcome fields (Appendix A) initialized to NONE/false, never left uninitialized
2 ;
  // CONFIRMED-NOT-SENT and UNKNOWN-EFFECT do not call an observation or confirmation interface that did not run
3 if  $er.\text{effect} \in \{\text{CONFIRMED-NOT-SENT}, \text{UNKNOWN-EFFECT}\}$  then
4    $z.\text{validity} \leftarrow \text{NONE}$ ; // an unset confirmation field is not an evaluated false
5 else
6    $z.\text{timing} \leftarrow \text{InterpretTimedObservation}(er, \xi, \Gamma, \eta, U)$ ;
7    $z.I_z \leftarrow \text{CompatibleInterpretation}(er, \xi, \Gamma)$ ;  $z.\text{summaryConfirmed} \leftarrow \text{true}$ ;  $z.\text{postSummary} \leftarrow$  confirmed successor
  // summary; // a normal TEST binds summaryConfirmed and postSummary explicitly
8   if  $er.\text{kind} \in \{\text{PREP}, \text{RECOVER}\}$  then
9      $z.\text{prepResultEvaluated} \leftarrow \text{true}$ ;
    ( $z.\text{targetConfirmed}$ ,  $z.\text{summaryConfirmed}$ ,  $z.\text{prepErr}$ ,  $z.\text{declaredTarget}$ ,  $z.\text{evidence}$ ,  $z.\text{postSummary}$ )  $\leftarrow$ 
    IF-PREP-RECOVER( $\Gamma, \eta, \xi.\text{actionId}, er.\text{record}$ );
    // IF-PREP-RECOVER returns values only; it does not write  $\Gamma$  or  $\eta$ ; a Prep whose target is unmet may
    // still have a confirmed successor
10 return  $z$ ; // values only; all  $\Gamma/\eta$  writes stay in ALG-CLTAV-04 and S9
```
